



## **PROYECTO**

### **ENTREGA 1**

Autor:

Diego Rueda

Marcos Pineda

Fátima Ramazzini

Catedrático:

Ing. Javier Álvarez

Guatemala 14 de agosto de 2026

## ÍNDICE

|                                         |          |
|-----------------------------------------|----------|
| <b>INTRODUCCIÓN</b>                     | <b>3</b> |
| <b>DIAGRAMA ENTIDAD-RELACIÓN</b>        | <b>3</b> |
| JUSTIFICACIÓN DE LA NORMALIZACIÓN:      | 4        |
| <b>CATÁLOGO DOCUMENTAL Y DECISIONES</b> | <b>4</b> |
| DIAGNÓSTICO DEL PROBLEMA                | 4        |
| MIGRACIÓN ETL                           | 4        |
| DECISIÓN: EMBEBER O REFERENCIAR         | 4        |
| Imágenes asociadas al producto          | 4        |
| Relación producto–vendedor              | 5        |
| Reseñas de producto                     | 5        |
| IMPLEMENTACIÓN FUNCIONAL DEL CATÁLOGO   | 6        |
| Índice compuesto                        | 6        |
| <b>ARQUITECTURA DEL SISTEMA</b>         | <b>7</b> |
| PUERTOS                                 | 7        |
| <b>FUTURAS MEJORAS</b>                  | <b>8</b> |

## ENTREGA 1

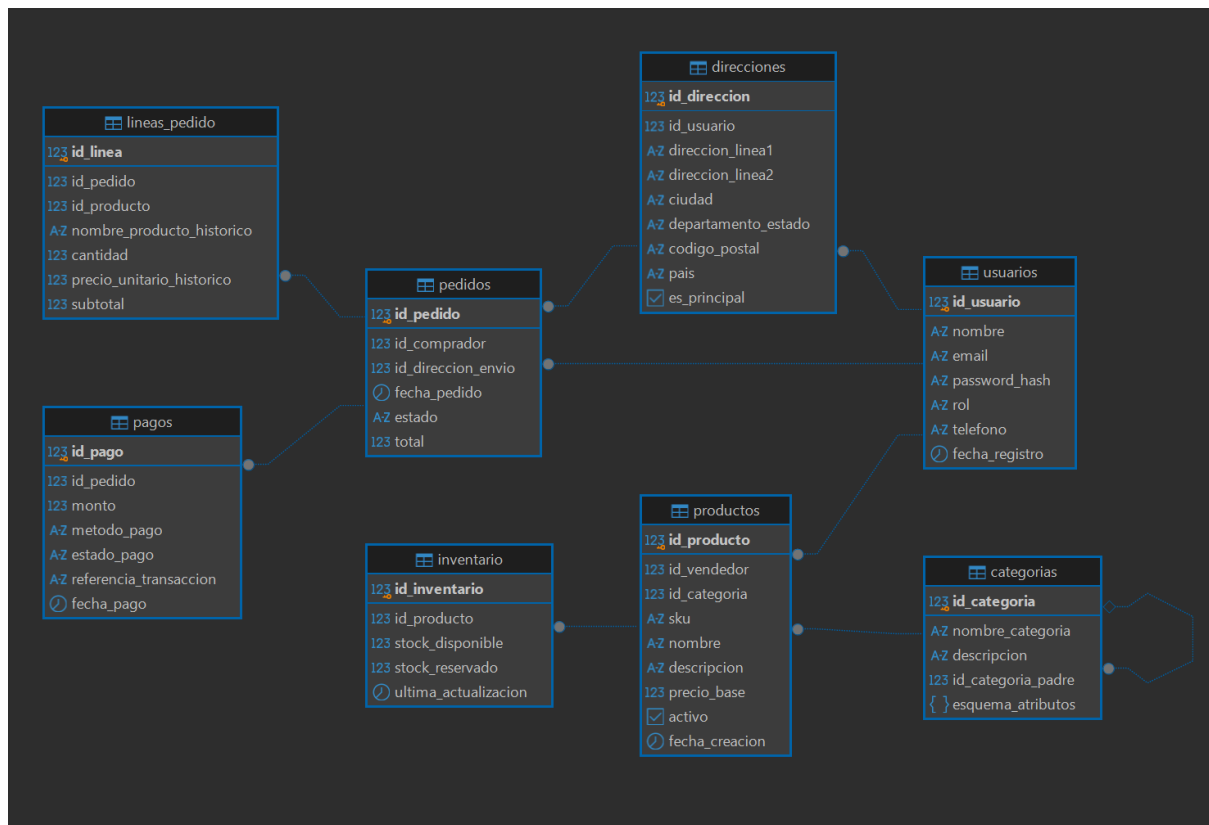
### INTRODUCCIÓN

TiendaYa es una plataforma de comercio electrónico construida como ejercicio de arquitectura de datos políglota: un núcleo transaccional en PostgreSQL (usuarios, pedidos, pagos, inventario) y un catálogo documental en MongoDB (productos con atributos heterogéneos por categoría e historial de cambios mediante event sourcing). Este informe cubre las dos primeras etapas del curso: la Fase Inicial (modelo relacional y checkout transaccional) y la Entrega 1 (migración del catálogo a un modelo documental).

### DATOS GENERALES

- Repositorio: [github.com/Fatiunis/proyectoBD2.0](https://github.com/Fatiunis/proyectoBD2.0)
- Stack:
  - PostgreSQL 18
  - MongoDB 8
  - Flask · JS

### DIAGRAMA ENTIDAD-RELACIÓN



### JUSTIFICACIÓN DE LA NORMALIZACIÓN:

- **Inventario separado de productos (1 a 1):** El stock cambia en cada compra; el resto de los atributos del producto casi nunca. Aislarlo en su propia tabla acota el patrón de escritura de alta frecuencia (UPDATE inventario) a la fila mínima necesaria.
- **Categorías auto-referenciadas:** id\_categoria\_padre modela una jerarquía (Tecnología → Laptops) sin duplicar taxonomías ni requerir una tabla de niveles fija.
- **Snapshot histórico intencional en líneas\_pedido.** nombre\_producto\_historico y precio\_unitario\_historico repiten datos que ya existen en productos. No es una violación de 3FN: una línea de pedido representa un hecho histórico inmutable (qué se cobró en ese momento), distinto del catálogo vigente, que sí cambia.

## CATÁLOGO DOCUMENTAL Y DECISIONES

### DIAGNÓSTICO DEL PROBLEMA

El catálogo relacional original enfrenta dos límites reales del modelo fijo: (1) categorías distintas necesitan atributos distintos — una laptop tiene RAM y GPU, una playera tiene talla y color — y forzarlos en columnas produce nulos masivos o una tabla EAV costosa de consultar; (2) un modelo que solo conserva el estado actual no puede responder "¿qué precio tenía este producto el 10 de agosto?", una pregunta legítima del negocio.

### MIGRACIÓN ETL

`database/migrations/migracion_postgres_a_mongo.py` extrae productos con un JOIN sobre categorías, vendedores e inventario, y escribe **dos colecciones: productos (catálogo) y historial\_cambios\_productos** (un evento CREACION\_PRODUCTO inicial por producto). El proceso es idempotente — recrea ambas colecciones desde cero en cada corrida — y reproducible con un solo comando (documentado en README.md). **Cada SKU semilla tiene su propio conjunto de atributos reales** (no una plantilla única por categoría), para que los filtros por atributo del catálogo tengan valores genuinamente distintos entre los que elegir.

### DECISIÓN: EMBEBER O REFERENCIAR

| Imágenes asociadas al producto |                                                                                                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto                       | Cada producto tiene pocas imágenes (portada + detalle), se consultan siempre junto con el producto, y cambian con poca frecuencia.                                 |
| Alternativas                   | (a) Colección imagenes aparte referenciada por producto_id.<br><b>(b) subarreglo embebido en el documento de producto.</b><br>(c) solo URLs, sin modelar en Mongo. |
| Decisión                       | <b>Subarreglo imagenes: [{id_imagen, url, es_portada, orden}] dentro del</b>                                                                                       |

|               |                                                                                                                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | <b>documento de producto.</b>                                                                                                                                                                                               |
| Justificación | Frecuencia de acceso conjunto del 100% (nunca se listan sin su producto), volumen bajo y acotado, tasa de cambio baja. Referenciar solo añadiría una consulta extra sin beneficio, ya que MongoDB no ofrece JOIN eficiente. |
| Consecuencias | Actualizar una imagen reescribe el documento completo (costo aceptable al volumen actual). Si las imágenes crecieran a decenas por producto, esta decisión debería revisarse,                                               |

| <b>Relación producto–vendedor</b> |                                                                                                                                                                                                                                                                 |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto                          | El catálogo debe mostrar el nombre del vendedor en cada carga sin consulta adicional; el perfil completo del vendedor vive en usuarios (PostgreSQL).                                                                                                            |
| Alternativas                      | (a) Referenciar id_vendedor y resolver el nombre contra PostgreSQL en cada lectura.<br><b>(b) embeber un snapshot mínimo.</b><br>c) duplicar el usuario completo en Mongo.                                                                                      |
| Decisión                          | <b>Snapshot mínimo embebido: {id_vendedor, nombre_comercial, email_contacto}</b>                                                                                                                                                                                |
| Justificación                     | El nombre del vendedor se lee en cada carga del catálogo — consultar PostgreSQL por producto listado introduciría una dependencia cruzada en el camino caliente de lectura. El snapshot es pequeño y cambia raramente.                                          |
| Consecuencias                     | Si un vendedor cambia su nombre comercial en PostgreSQL, el catálogo en Mongo queda desactualizado hasta que cada producto se vuelva a guardar — inconsistencia eventual aceptada explícitamente en esta etapa. No se duplican credenciales ni datos sensibles. |

| <b>Reseñas de producto</b> |                                                                                                                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto                   | Aún no implementadas (no son funcionalidad de Fase Inicial ni Entrega 1); se deja la decisión de modelado tomada de antemano.                                                                                       |
| Alternativas               | (a) Arreglo de reseñas embebido en el documento de producto.<br><b>(b) colección reseñas aparte referenciada por producto_id.</b>                                                                                   |
| Decisión                   | <b>Colección aparte, referenciada</b>                                                                                                                                                                               |
| Justificación              | Volumen no acotado (un producto popular puede acumular miles) y alta tasa de escritura independiente del producto. Embeberlas arriesgaría el límite de 16MB por documento y degradaría cada escritura del catálogo. |

|               |                                                                                                                                                                                                         |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | Entrega 2 exige además detectar patrones anómalos entre cuentas y reseñas — un análisis que requiere que sean su propia entidad consultable.                                                            |
| Consecuencias | Listar las reseñas de un producto exige una consulta adicional, pero es el patrón correcto: las reseñas también se consultan de forma independiente (por usuario, por fecha, para detección de fraude). |

## IMPLEMENTACIÓN FUNCIONAL DEL CATÁLOGO

### Índice compuesto

Sobre el patrón de consulta real del catálogo (filtrar por categoría y disponibilidad, ordenar por precio):

```
db.productos.createIndex(
    { "categoria.id_categoria": 1, "activo": 1, "precio_base": 1 },
    { name: "idx_categoria_activo_precio" }
);
```

### Marco de agregación

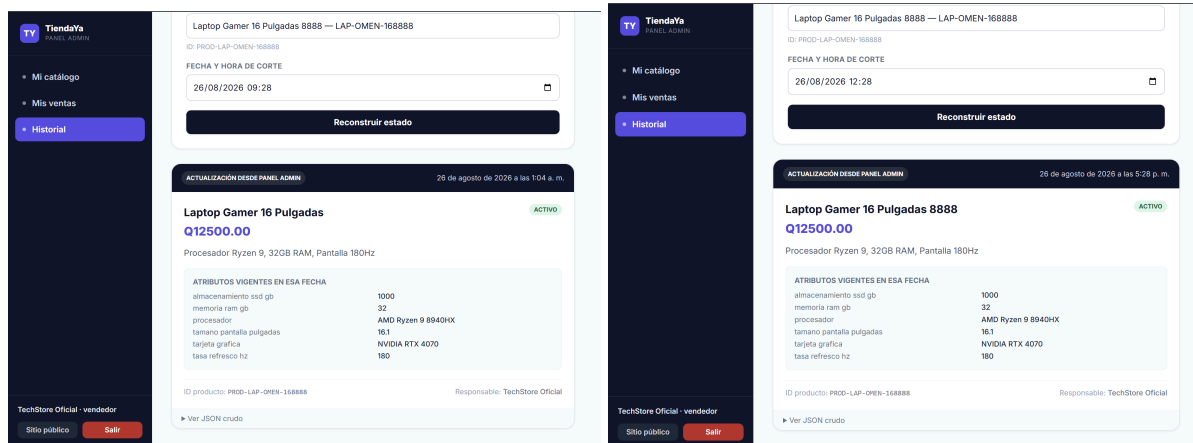
Esta consulta extrae y agrupa todos los atributos de los productos de una categoría específica para poder construir dinámicamente los filtros de búsqueda en la interfaz de la tienda.

```
def get_filtros_categoria(id_categoria):
    pipeline = [
        {"$match": {"categoria.id_categoria": id_categoria, "activo": True}},
        {"$project": {"pares": {"$objectToArray": "$atributos"}}},
        {"$unwind": "$pares"},
        {"$group": {"_id": "$pares.k", "valores": {"$push": "$pares.v"}}}
    ]
    grupos = list(col_productos.aggregate(pipeline))
```

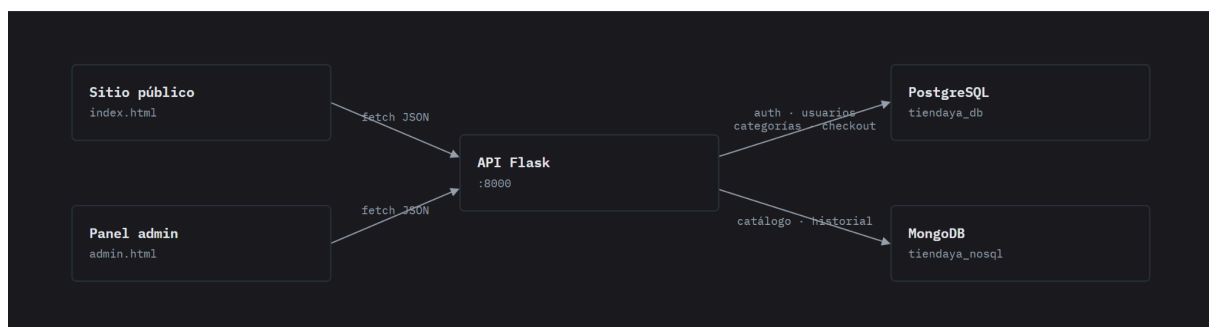
El backend descarta como filtro cualquier atributo constante entre productos (no discrimina nada) y cualquier atributo de texto donde cada producto tiene un valor único (ej. procesador, que no se ofrece como filtro por esta razón).

## HISTORIAL DE CAMBIOS

Cada modificación de un producto agrega un documento nuevo a historial\_cambios\_productos — nunca se sobrescribe el estado anterior. GET /api/historial/<id>?fecha\_corte=... reconstruye el estado vigente en cualquier fecha con una agregación \$match + \$sort + \$limit 1.



## ARQUITECTURA DEL SISTEMA



## PUERTOS

PG\_HOST=localhost

PG\_PORT=5432

PG\_DBNAME=tiendaya\_db

PG\_USER=postgres

PG\_PASSWORD=root \*propia

MONGO\_URI=mongodb://localhost:27017/

MONGO\_DB\_NAME=tiendaya\_nosql

### **FUTURAS MEJORAS**

- Historial de producto que se muestre automáticamente los últimos cambios del mismo modo cuando se selecciona un producto. Implementación de un filtro para poder obtener datos por rango de fecha
- Al cerrar sesión un admin o vendedor la url a la que debe redirigir es la de index y no admin para poder iniciar con cualquier rol de usuario.